Computer Architecture Lecture Notes

Scratch-pad memory, Cache Coherence, and Prefetching

Varukolu Vishal CS23BTECH11064
Asish Mourya Birudu CS23BTECH11010
Harshith Peddineni CS23BTECH11045
Shriiraz Kukatla CS23BTECH11029

28 October 2024

Contents

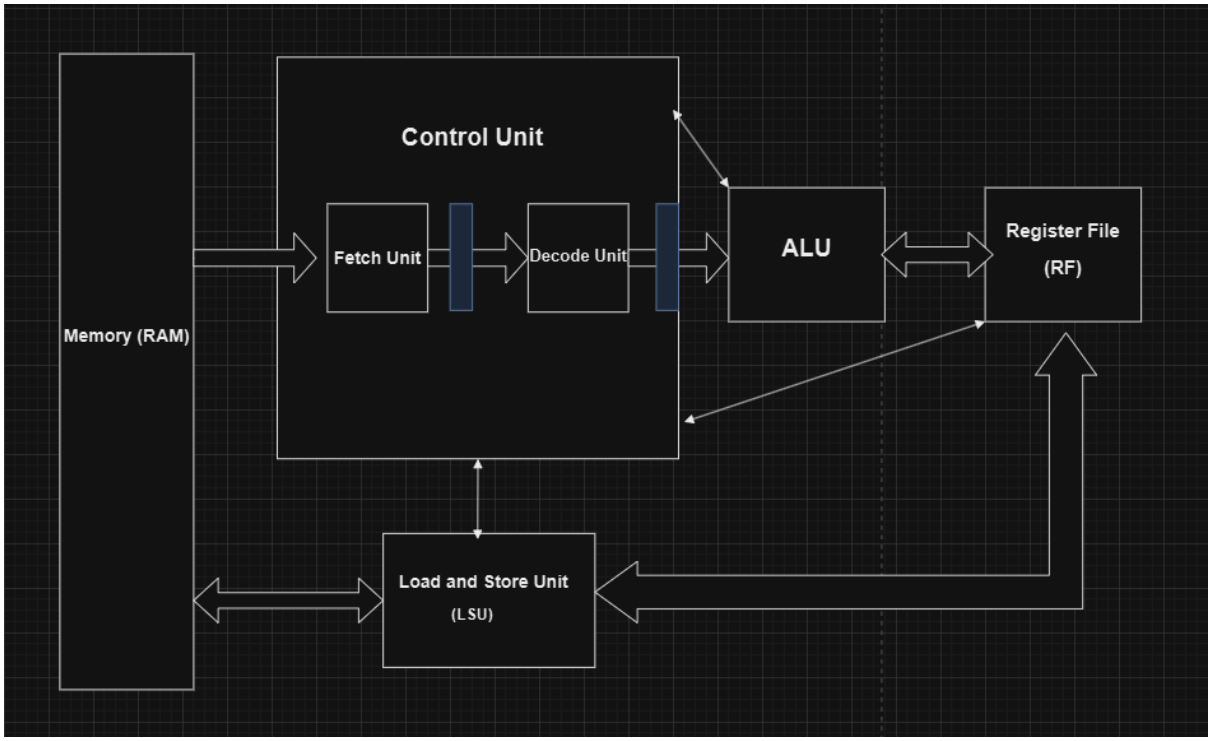
1 Overview of Caches 2
1.1 Until Now (in Caches) 2
2 Processor-Memory Interface (without cache) 2
3 Detailed Processor Memory Interface (with cache) 3
4 Scratch Pad Memory (SPM) 3
5 Caches in RED-V Board used in experiments 4
6 Cache Coherence Problem 5
7 Cache Coherence Protocols 5
8 Invalidating Snooping Protocols 6
9 Prefetching 7
10 Cache Summary 8
11 System Optimizations 9
12 Concluding Remarks 9

Overview of Caches

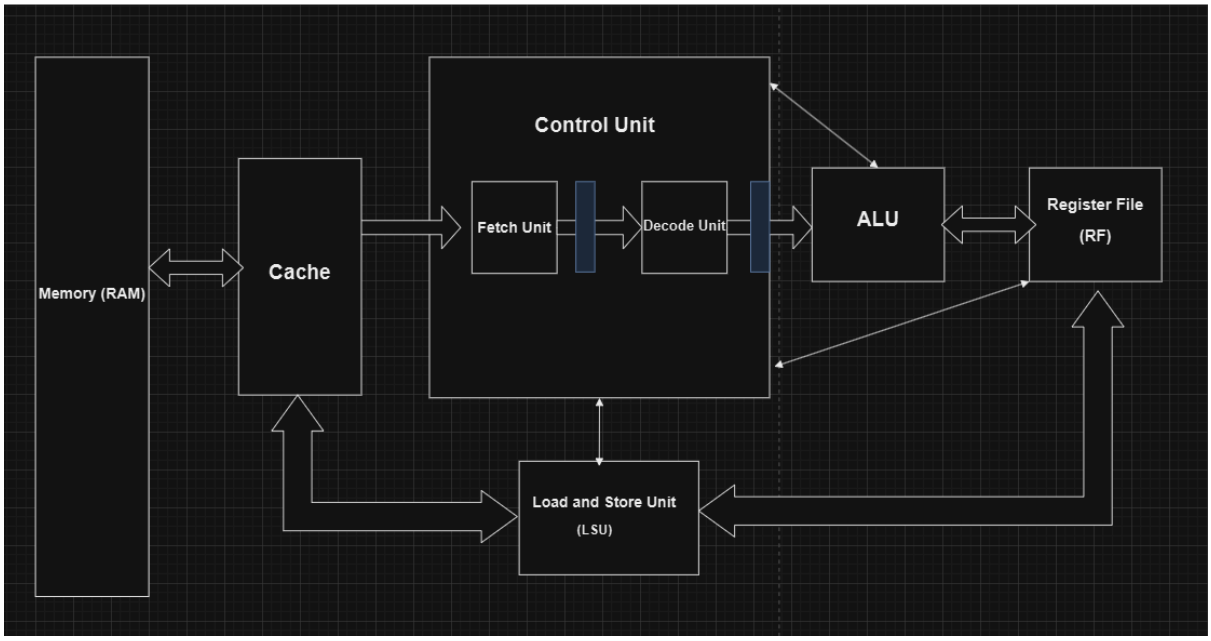
1.1 Until Now (in Caches)

- **Memory Wall:** Locality behavior of programs.
- **Cache:** A faster, smaller memory placed near CPU.
  - Components: Tag, Index, and Offset bits.
  - Structure: Dividing a cache into sets and ways.
- **Key Concepts:**
  - Associativity, Block size, Capacity.
  - Write policies: Write-through and Write-back.
  - Replacement policies: FIFO, LRU, Random, LFU, Combinations.
  - Types of misses: Compulsory, Capacity, Conflict (the 3 C's).
  - Impact of design choices (Associativity, Block size, Capacity) on misses.
  - Write-buffer to reduce write time overheads.
  - Performance calculation: Stall cycles, Average memory access time.
  - Cache architectures: Unified versus split cache.
  - Multi-level cache: L1I + L1D + L2 + (L3).
  - Last level Caches (LLC) are L2/L3 in most systems
  - Cache overheads: Area, Delay, Power, Unpredictable timing.

Processor-Memory Interface (without cache)



Detailed Processor Memory Interface (with cache)



Scratch Pad Memory (SPM)

- **Caches and Timing Uncertainty:** While caches can significantly speed up memory access by reducing latency, they inherently introduce unpredictability in execution times due to factors such as cache misses and varying levels of associativity. The same code might take different execution times depending upon whether accesses encounter hits or misses during execution.
- Many embedded systems, especially those used in real-time applications like multimedia, automotive systems, etc. require strict timing guarantees. In these systems, variability in execution time is undesirable as it may lead to missed deadlines or unstable performance.
- **Overhead and Limitations of Caches in Embedded Systems:** Smaller embedded systems may also lack the computational resources or budget to support the overhead of managing cache hardware, leading designers to seek alternative methods to maintain predictable performance.

Solution: Utilizing Scratch Pad Memory (SPM) as an alternative to Caches

- **DEFINITION:** Unlike caches which automatically store recently used data and handle entry/exit of data within cache, SPM is a manually managed on-chip SRAM (Static Random Access Memory) where data is explicitly stored or removed by the software program. In a way, a SPM acts as a "software-managed cache". Unlike traditional hardware caches, which operate transparently, SPM requires explicit control by the software, giving developers more direct control over memory access patterns.
- **Physical Location and Speed Advantages:** SPM is typically placed close to the CPU on the chip, making it faster to access than off-chip DRAM. It provides reduced access latency, ensuring that critical data and instructions can be retrieved more quickly than if they were in main memory.

- **Software-Managed Data Transfer:**
  - Software must explicitly handle the transfer of data between DRAM and SPM. For example, a developer might write routines to copy necessary data blocks from DRAM to SPM at program startup or whenever the data becomes relevant.
  - This software-driven data management allows more predictable timing, as the access patterns to SPM can be controlled and planned by the programmer rather than relying on dynamic cache mechanisms.
  - SPM can store either instructions or data, and it is entirely upto the associated software on how it plans to use SPM. Unlike cache where some useful data can get evicted due to some other block needing that set, a piece of data from SPM can be removed only by the software.
- **Use Cases for SPM:** SPM can store frequently accessed data and functions that are critical to performance, such as:
  - Commonly used routines (e.g., mathematical functions in signal processing, critical interrupt handlers, etc.).
  - Global variables or data structures frequently accessed by various parts of the program.
  - Loop bodies or critical sections of code that benefit from fast, predictable access.
- **Data Management Strategies:**
  - **Static Allocation:** Data is preloaded into SPM at the start of execution. This is useful for routines that are known to be critical throughout program execution.
  - **Dynamic Allocation:** Data is copied to SPM as needed during program execution, often based on access patterns or program state. This approach is more flexible but requires careful planning to ensure that data remains available when needed.
  - Compilers can automatically decide which data to be brought into SPM at what time based on access pattern analysis or can be identified and implemented by the programmer explicitly.
- **Hybrid Memory System:** Some systems combine SPM with traditional cache to take advantage of both:
  - Critical routines and frequently accessed data are placed in SPM for predictable, low-latency access.
  - Non-critical or less frequently accessed data can reside in cache, benefiting from cache's automated management but tolerating some timing variability.
  - This hybrid approach allows a balance between deterministic performance for real-time requirements and flexibility for less timing-sensitive operations.

Caches in RED-V Board used in experiments

- Features the FE310 SoC from SiFive.
- Supports the RV32IMAC instruction set.
- Data Tightly Integrated Memory (DTIM) subsystem includes 16 KB of Scratch Pad Memory.
- 16 KB 2-way instruction cache with a line size of 32 Bytes
  - 16 KB I-cache can also be configured as 8 KB of I-cache + 8 KB of SPM called as Instruction Tightly Integrated Memory (ITIM).

Cache Coherence Problem

Consider a scenario where two CPU cores share a physical address space:

Caches can lead to non-coherence behaviour as the other CPU core may not be aware of the other core getting updated. Let's look at an example for a write-through cache.

Time	Event	CPU A's Cache	CPU B's Cache	Memory
0				0
1	CPU A reads X	0		0
2	CPU B reads X	0	0	0
3	CPU A writes 1 to X	1	**0**	1

Coherence Definition

- **Informally:** Reads return the most recently written value.
- **Formally:**
  - If CPU P writes X and P reads X (no intervening writes) $\Rightarrow$ read returns the written value.
  - If CPU P_1 writes X and CPU P_2 reads X (sufficiently later) $\Rightarrow$ read returns the written value (e.g., CPU B reading X after step 3 in the previous example).
  - If P_1 writes X and P_2 writes X, all processors see writes in the same order and end up with the same final value for X.

Cache Coherence Protocols

Cache coherence protocols are essential in multiprocessor systems, ensuring that multiple processors see consistent values for shared memory. They prevent scenarios where different processors have outdated or conflicting data by coordinating updates across caches.

- **Purpose:** These protocols define how caches maintain a consistent view of shared data, ensuring that updates are propagated to all caches when one cache modifies a value.
- **Data Migration:** Coherence protocols enable frequently accessed data to be stored in local caches, reducing memory bandwidth usage and improving latency for data access.
- **Data Replication:** When multiple processors read the same data, coherence protocols allow for its replication in local caches, minimizing contention and enabling faster access.
- **Snooping Protocols:**
  - Caches monitor the shared bus for read/write operations. When a cache modifies a block, it broadcasts a message to inform other caches to invalidate or update their copies.
  - **Advantages:** Simple and effective for small systems, enabling quick updates.
  - **Limitations:** Performance degrades with larger number of processors due to bus contention and increased broadcast traffic.
- **Directory-Based Protocols:**
  - A central directory keeps track of which caches hold copies of each memory block, reducing the need for constant broadcasting.
  - **Advantages:** More scalable than snooping protocols, suitable for larger systems.
  - **Limitations:** Complexity and potential bottlenecks due to the overhead of maintaining the directory.

Invalidating Snooping Protocols

- **Exclusive Access:** A cache must gain exclusive access to a memory block before modifying it, ensuring no other cache has an outdated copy during the update.
- **Invalidate Message:** After securing exclusive access, the cache broadcasts an invalidate message, notifying other caches that their copies of the block are no longer valid.
- **Subsequent Reads:** If another cache tries to read an invalidated block, it incurs a cache miss. The modified cache then supplies the updated value, ensuring consistency.
- **MSI Protocol (3 states for a cache block)**
  - **Invalid:** The block is not valid and needs to be fetched.
  - **Shared (Read-Only):** The block is valid and can be accessed by multiple caches without modification. This block has not been modified by any cache yet.
  - **Modified (Exclusive-Write):** The block has been changed by the current cache, and only this cache has the most recent copy.
- **Write-Invalidate vs. Write-Update:**
  - **Write-Invalidate:** The modifying cache sends an invalidate signal to others, minimizing bus traffic.
  - **Write-Update:** The new value is broadcasted to all caches, ensuring immediate consistency at the cost of increased bus traffic.
- **False Sharing:** When block size is very large, it may happen that different cores are updating different bytes of this block. Even though they are not updating the same locations, it might seem like same block getting updated and can lead to cache system think of it as a false shared data. This leads to unnecessary invalidations and performance issues.

Table 1: Multilevel On-Chip Caches, an example

Characteristic	ARM Cortex-A53	Intel Core i7
L1 Cache Organization	Split instruction and data caches	Split instruction and data caches
L1 Cache Size	Configurable 16 to 64 KB each for instructions/data	32 KB each for instructions/data per core
L1 Cache Associativity	Two-way (I) Four-way (D) set associative	Four-way (I) Eight-way (D) set associative
L1 Replacement	Random	Approximated LRU
L1 Block Size	64 bytes	64 bytes
L1 Write Policy	Write-back, variable allocation (default is Write-allocate)	Write-back, No-write-allocate
L1 Hit Time (load+use)	2 clock cycles	4 clock cycles, pipelined
L2 Cache Organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 Cache Size	128 KiB to 2 MiB	256 KiB (0.25 MiB)
L2 Cache Associativity	16-way set associative	8-way set associative
L2 Replacement	Approximated LRU	Approximated LRU
L2 Block Size	64 bytes	64 bytes
L2 Write Policy	Write-back, Write-allocate	Write-back, Write-allocate
L2 Hit Time	12 clock cycles	10 clock cycles
L3 Cache Organization	–	Unified (instruction and data)
L3 Cache Size	–	8 MiB, shared
L3 Cache Associativity	–	16-way set associative
L3 Replacement	–	Approximated LRU
L3 Block Size	–	64 bytes
L3 Write Policy	–	Write-back, Write-allocate
L3 Hit Time	–	35 clock cycles

Prefetching

Prefetching further reduces cache miss penalties by proactively loading data into the cache before it is requested by the CPU. Typically, a block is loaded into cache only upon its first access, causing a miss penalty for the first miss. Prefetching anticipates future accesses and fetches the block upfront, improving performance by minimizing these misses.

- **Need for Prefetching:** When data is accessed for the first time, it incurs a cache miss as it's fetched from memory. Prefetching aims to load data beforehand, reducing this delay.
- **Effective Access Patterns:** Prefetching is especially useful for predictable patterns:
  - **Sequential Access:** Prefetching anticipates consecutive accesses, minimizing misses.
  - **Strided Access:** Fetching data at regular intervals can benefit from targeted prefetching.
  - **Irregular Access:** Limited or no gains from prefetching, as patterns are less predictable.
- **Hardware Prefetching:** Hardware prefetchers dynamically fetch data based on observed patterns, needing no software intervention. Although automatic, they can sometimes fetch not-needed

data or may not be able to identify future accesses better, causing cache pollution and wastage of memory bandwidth.

- **Software Prefetching:** With software prefetching, the programmer or compiler inserts "prefetch" instructions to bring data into the cache as needed. This approach offers better prefetch accuracy but requires clear knowledge of access patterns to avoid unnecessary overhead.
- Effective prefetching boosts cache performance but may waste resources if overused. Modern processors use adaptive prefetching strategies, balancing hardware and software methods to optimize cache utilization.

Cache Summary

- **Caches help reduce memory access latency.**
- **Types of Caches:**
  - Instruction Cache
  - Data Cache
  - Unified Cache (Instruction + Data)
- **Locality:**
  - Spatial Locality
  - Temporal Locality
- **Cache Structure Components:** Tag, Index, and Offset.
- **Types of Cache Organizations:**
  - Direct Mapped (Associativity of 1)
  - Set Associative
  - Fully Associative
- **Write Policies:**
  - Write-Back
  - Write-Through
- **On Cache Miss:** Allocate or No-Allocate strategies.
- **Replacement Policies:** Random, FIFO, LRU, LFU, Mixed.
- **Types of Cache Misses:** Compulsory, Conflict, and Capacity.
- **Cache Overheads:** Time, Area, and Power considerations.
- **Multi-Level Cache:** Further reduces access latency.
- **Scratch Pad Memory (SPM):** Provides predictable timing in embedded systems.
- **Prefetching:** Enhances performance through proactive data loading.
- **Cache Parameters:** Key factors include Associativity, Block size, and Cache size/Capacity.
- **Write Policy:** Influences the selection of replacement policy.

System Optimizations

- **Loop Optimizations:**
  - Loops can cause pipeline hazards due to branch instructions.
  - Unrolling a loop reduces branch overhead and enables better usage of registers.
  - But it impacts on performance as large amount of unrolling can lead the instructions overflow from the instruction cache and hurt instruction cache performance due to increased cache misses.
- **Function Optimizations:**
  - Functions may introduce pipeline hazards due to jump instructions.
  - They can break spatial locality, affecting cache performance.
  - Additional instructions are required to save and restore context.
  - Function inlining is a technique used by compilers – but inlining can increase the overall code size and negatively affect the temporal locality and increase I-cache misses.
- **Array Access Order:**
  Row-wise access pattern is generally more efficient than column-wise in the following ways:
  - Row-wise access uses entire cache lines whereas column-wise may use only one element per cache line.
  - column-access can cause cache misses and Hardware prefetchers can predict and load next row elements.
  - The performance difference is largely noticable and increases with larger arrays.
  - Languages like MATLAB use column-major ordering where column-wise access would be more efficient.

Concluding Remarks

- Fast memories tend to be small, while larger memories are often slower.
- The ideal scenario is to have both fast and large memories – Caching techniques create the illusion of this capability.
- **Principle of Locality:** Programs typically access a small portion of their memory space frequently.
- **Memory Hierarchy:** L1 cache $\leftrightarrow$ L2 cache $\leftrightarrow \ldots \leftrightarrow$ DRAM memory $\leftrightarrow$ disk storage.
- Effective memory system design is critical for performance.
- Cache-aware code optimizations are essential for maximizing efficiency.